

Préprocessing & feature engineering

Préparer les données avant le modèle, sans fuite · Marianne A. · compagnon de M1

Le modèle ne voit que ce qu'on lui donne. Préparer les données pèse souvent plus lourd dans le résultat que le choix de l'algorithme. Un bon préprocessing transforme des colonnes brutes (manquants, échelles disparates, catégories en texte) en un tableau numérique propre que le modèle peut exploiter.

Règle d'or (rappel M1). Tout le préprocessing va dans un `Pipeline`, appris avec `.fit` sur le **TRAIN seulement**, puis appliqué (`.transform`) au test. Sinon le test « fuit » dans la préparation et le score devient mensonger.

Règle `fit sur train` → `transform sur test`, jamais l'inverse.

1 Le pipeline de préparation

La préparation suit un enchaînement stable, qu'on assemble une fois pour toutes :

1) Identifier les types de colonnes — numérique, catégoriel, date, texte ; chaque type appelle un traitement différent. **2) Imputer les manquants** — remplir les trous avant tout le reste, car la plupart des transformateurs et modèles refusent les NaN. **3) Transformer le numérique** — mettre à l'échelle (*scaling*) pour que les variables soient comparables. **4) Encoder le catégoriel** — convertir les modalités texte en nombres. **5) Features dérivées éventuelles** — extraire de l'information (jour de semaine d'une date, ratios, encodage cyclique...). **6) Modèle** — le classifieur/régresseur en dernier maillon.

Le tout s'orchestre avec un **ColumnTransformer** (qui applique un traitement différent selon les colonnes) enveloppé dans un **Pipeline** qui se termine par le modèle. On appelle `.fit` une seule fois sur `X_train` : toutes les statistiques (médianes, moyennes, écarts-types, modalités connues) sont apprises là, puis rejouées à l'identique sur le test.

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

num_cols = ["age", "revenu", "anciennete"]
cat_cols = ["ville", "offre"]

num_pipe = Pipeline([
    ("imp", SimpleImputer(strategy="median")), # manquants num → médiane
    ("sc", StandardScaler()),                # mise à l'échelle
])
cat_pipe = Pipeline([
    ("imp", SimpleImputer(strategy="most_frequent")),
    ("oh", OneHotEncoder(handle_unknown="ignore")), # catégorie inconnue → 0
])

pre = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols),
])

pipe = Pipeline([
    ("pre", pre),
    ("clf", RandomForestClassifier(random_state=42)),
])

pipe.fit(X_train, y_train) # .fit appris sur le TRAIN uniquement
pipe.predict(X_test)      # le préprocessing est rejoué à l'identique

```

2 Traiter chaque type de variable

TYPE	TRANSFORMATION	QUAND / À SAVOIR
Numérique	StandardScaler (défaut), MinMaxScaler (bornes connues), RobustScaler (outliers).	Utile pour les modèles linéaires et à distance (kNN, SVM). Les arbres n'en ont pas besoin.

TYPE	TRANSFORMATION	QUAND / À SAVOIR
Catégorielle nominale	OneHotEncoder(handle_unknown="ignore").	Pas d'ordre entre les modalités : une colonne binaire par modalité.
Catégorielle ordinale	OrdinalEncoder avec ordre explicite (<i>faible < moyen < fort</i>).	Quand les modalités ont un ordre naturel qu'on veut préserver.
Haute cardinalité	Frequency ou target encoding.	Attention fuite : encoder dans la CV , jamais sur tout le jeu.
Date	Extraire année / mois / jour / jour de semaine ; encodage cyclique sin/cos pour mois & heure.	Le cyclique évite la fausse cassure entre décembre et janvier (ou 23h et 0h).
Texte	TF-IDF (classique) ou <i>embeddings</i> (→ deep learning).	TF-IDF suffit souvent ; les embeddings capturent le sens mais coûtent plus.

3 Valeurs manquantes

STRATÉGIE	COMMENT	QUAND
Supprimer lignes/colonnes	Retirer les lignes ou colonnes trouées.	Si très rares. Attention à ne pas perdre de signal.
Imputer simple	SimpleImputer — médiane (num), mode ou constante (cat).	Cas courant, robuste et rapide.
Imputer avancé	KNNImputer, IterativeImputer.	Si les manquants portent de l'information exploitable.
Indicateur de manquant	Ajouter une colonne était_manquant (0/1).	Si la <i>manquance elle-même</i> est informative.

Note. Imputer **avant** le split = fuite (la médiane/mode a « vu » le test). Toujours mettre l'imputation **dans le pipeline** pour qu'elle soit apprise sur le train et rejouée sur le test.

4 Pièges fréquents

LE PIÈGE	LE CORRECTIF
Préprocessing appris sur tout X (avant le split) → fuite.	Tout dans le Pipeline, .fit sur le train seul.
Encoder une info dérivée de la cible dans les features → <i>target leakage</i> .	Bannir : aucune feature ne doit contenir d'info issue de y.

LE PIÈGE	LE CORRECTIF
OneHot sur très haute cardinalité → explosion de colonnes.	Frequency ou target encoding (encodé dans la CV).
Catégorie inconnue au test → crash.	OneHotEncoder(handle_unknown="ignore").
Standardiser inutilement les arbres.	Sans effet : inutile, mais pas nuisible.
Oublier de gérer les manquants.	Erreur au .fit : prévoir un imputer dans le pipeline.

La règle d'or. types → imputation → scaling/encodage : **tout** dans un Pipeline, .fit sur le train uniquement. Ainsi le preprocessing *voyage avec le modèle* — on sauvegarde l'ensemble avec joblib et il se rejoue à l'identique en production.

À retenir. La préparation des données décide souvent plus que l'algorithme : identifier les types, imputer, mettre à l'échelle et encoder — le tout dans un pipeline appris sur le seul train — pour **Fiche 3/12 · Série ML** un score honnête et un modèle réutilisable.